Bitcoin Bootcamp

Student Setup Guide

A step-by-step guide to setting up Bitcoin Core (Regtest), Polar Lightning, and Python on macOS, Linux, and Windows.

Version	1.0 — March 2026
Bitcoin Core	Built from source (CMake)
Polar	Latest release
Python	3.10+

Table of Contents

1. Hardware Requirements

Before you begin, make sure your laptop meets the following minimum specifications. Since we are using **regtest mode** (a local test blockchain), the requirements are much lighter than running a full mainnet node.

Component	Minimum	Recommended
CPU	Dual-core 1.5 GHz	Quad-core 2.0 GHz+
RAM	4 GB	8 GB+
Free Disk Space	5 GB	10 GB+
OS	macOS 11+, Ubuntu 20.04+, Windows 10+	Latest stable release
Internet	Required for downloads	Stable broadband

Note: For **Plan B (Polar Lightning)**, Docker is required and tends to use more RAM. If your machine has only 4 GB of RAM, close other applications while running Polar.

2. Plan A — Building Bitcoin Core from Source (Regtest)

Bitcoin Core is the reference implementation of the Bitcoin protocol. In this bootcamp we use **regtest** (regression test) mode, which creates a private local blockchain that you fully control. You can mine blocks instantly, create transactions, and experiment without needing to sync the real blockchain or spend real bitcoin.

We will **build Bitcoin Core from source** rather than using pre-built binaries. This gives you a deeper understanding of the software and ensures you have the latest version. The build system uses **CMake** (replacing the older Autotools system as of v29+).

> **Note:** The official build documentation lives at: **https://github.com/bitcoin/bitcoin/blob/master/doc/** — see build-unix.md, build-osx.md, and build-windows.md for full details.

2.1 macOS — Build from Source

macOS

Step 1: Install Xcode Command Line Tools

```
xcode-select --install
```

Step 2: Install Homebrew (if not already installed)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 3: Install dependencies

```
brew install cmake boost pkgconf libevent
```

> **Note:** SQLite is required for wallet support but macOS ships with a usable sqlite package, so no extra install is needed.

Step 4: Clone the repository

```
git clone https://github.com/bitcoin/bitcoin.git
cd bitcoin
```

Step 5: Build

```
cmake -B build
cmake --build build -j $(sysctl -n hw.ncpu)
```

Once complete, the binaries are at `./build/bin/bitcoind`, `./build/bin/bitcoin-cli`, etc.

Step 6 (optional): Install system-wide

```
sudo cmake --install build
```

This copies the binaries to `/usr/local/bin/` so they are on your PATH.

2.2 Linux (Ubuntu/Debian) — Build from Source

Linux

Step 1: Install build dependencies

```
sudo apt-get update
sudo apt-get install -y build-essential cmake pkgconf python3 \
libevent-dev libboost-dev libsqlite3-dev
```

Step 2: Clone the repository

```
git clone https://github.com/bitcoin/bitcoin.git
cd bitcoin
```

Step 3: Build

```
cmake -B build
cmake --build build -j $(nproc)
```

Step 4 (optional): Install system-wide

```
sudo cmake --install build
```

2.3 Windows — Build via WSL (Recommended)

Windows

For Windows, we **strongly recommend** using the Windows Subsystem for Linux (WSL). WSL gives you a full Linux environment inside Windows, making it much easier to compile and run Bitcoin Core. All terminal commands will run inside the WSL Ubuntu shell.

Step 1: Install WSL and Ubuntu

Open **PowerShell as Administrator** and run:

```
wsl --install
```

This installs WSL 2 with Ubuntu by default. Restart your computer when prompted. After reboot, Ubuntu will open and ask you to create a username and password.

> **Note:** If WSL is already installed, you can install Ubuntu specifically with:
> `wsl --install -d Ubuntu`

Step 2: Update Ubuntu packages

Inside your WSL Ubuntu terminal:

```
sudo apt-get update && sudo apt-get upgrade -y
```

Step 3: Install build dependencies

```
sudo apt-get install -y build-essential cmake pkgconf python3 \
libevent-dev libboost-dev libsqlite3-dev git
```

Step 4: Clone the repository

> **Warning:** The source code **must** be on the Linux filesystem (e.g. `/home/youruser/` or `/usr/src/`), **NOT** under `/mnt/c/` or `/mnt/d/`. Building on the Windows mount will cause failures.

```
cd ~
git clone https://github.com/bitcoin/bitcoin.git
cd bitcoin
```

Step 5: Build

```
cmake -B build
cmake --build build -j $(nproc)
```

Step 6 (optional): Install system-wide inside WSL

```
sudo cmake --install build
```

> **Note:** After building, you will run `bitcoind` and `bitcoin-cli` from inside the WSL terminal for the rest of the bootcamp. The Bitcoin data directory will be at `~/.bitcoin/` inside WSL.

2.4 Configuring Regtest Mode

macOS Linux Windows

Create a configuration file so Bitcoin Core starts in regtest mode by default. The data directory and config path differ by platform.

Platform	Config File Location
macOS	`~/Library/Application Support/Bitcoin/bitcoin.conf`
Linux	`~/.bitcoin/bitcoin.conf`
Windows (WSL)	`~/.bitcoin/bitcoin.conf  (inside WSL)`

Open (or create) the file and add the following lines:

```
# Bitcoin Core — Regtest configuration
regtest=1
server=1
rpcuser=bootcamp
rpcpassword=bootcamp123
rpcport=18443
fallbackfee=0.00001
[regtest]
rpcbind=127.0.0.1
rpcallowip=127.0.0.1
```

Start bitcoind in regtest mode:

```
bitcoind -regtest -daemon
```

Or, if you added `regtest=1` to your config:

```
bitcoind -daemon
```

Note: If you did not run `cmake --install`, use the full path from your build directory:
`./build/bin/bitcoind -regtest -daemon`

2.5 Verifying Your Installation

macOS Linux Windows

Run the following commands to confirm everything works:

```
# Check the version
bitcoin-cli -regtest --version

# Get blockchain info (should show regtest chain)
bitcoin-cli -regtest getblockchaininfo

# Create a test wallet
bitcoin-cli -regtest createwallet "bootcamp_wallet"

# Generate an address
bitcoin-cli -regtest getnewaddress

# Mine 101 blocks (makes coins spendable)
bitcoin-cli -regtest generatetoaddress 101 $(bitcoin-cli -regtest getnewaddress)

# Check your balance
bitcoin-cli -regtest getbalance
```

Note: You should see a balance of 50 BTC (the block reward from the first matured coinbase).

3. Plan B — Installing Polar Lightning

Polar is a desktop application that lets you spin up a local Bitcoin and Lightning Network environment with a single click. It runs Bitcoin Core, LND, CLN (Core Lightning), and Eclair nodes inside Docker containers, giving you a visual interface for managing channels and payments.

When to use Plan B: If you had trouble with the manual Bitcoin Core installation, or if the bootcamp exercises focus on Lightning Network development, Polar provides a faster and more visual setup experience.

3.1 Prerequisites: Docker

Polar requires Docker to run its containerised nodes. Install Docker first.

macOS and Windows

macOS Windows

- Download and install **Docker Desktop** from **https://www.docker.com/products/docker-desktop/**
- Run the installer and follow the prompts.
- After installation, open Docker Desktop and make sure it shows **"Docker is running"** in the bottom-left.

> **Note:** On macOS with Apple Silicon (M1/M2/M3/M4), Docker Desktop runs Linux containers through Rosetta. No special setup is needed.

Linux

Linux

Install Docker Engine and Docker Compose:

```
# Update packages
sudo apt update

# Install Docker
sudo apt install -y docker.io docker-compose-v2

# Add your user to the docker group (avoids needing sudo)
sudo usermod -aG docker $USER

# Log out and back in, then verify
docker --version
docker compose version
```

Warning: You **must** log out and log back in after adding yourself to the docker group, otherwise Polar will fail to start nodes.

3.2 Installing Polar

macOS Linux Windows

- Go to **https://lightningpolar.com/** and click the download button for your operating system.
- Alternatively, download from GitHub: **https://github.com/jamaljsr/polar/releases**

Platform	File to Download	Install Steps
macOS	Polar-x.x.x.dmg	Open the .dmg file, drag Polar to Applications.
Linux	Polar-x.x.x.AppImage	Make it executable: chmod +x Polar-*.AppImage Then double-click or run from terminal.
Windows	Polar-x.x.x-Setup.exe	Run the installer and follow the prompts.

3.3 Creating Your First Network

Once Polar is installed and Docker is running:

- **Step 1:** Open Polar. On first launch, it may pull Docker images — this can take a few minutes.
- **Step 2:** Click **"Create Network"** and give it a name (e.g., "Bootcamp").
- **Step 3:** Choose your node implementations. The defaults are fine: 1 Bitcoin Core node and 2 LND nodes.
- **Step 4:** Click **"Start"**. Polar will spin up the Docker containers.
- **Step 5:** Once all nodes show a green status, your local Lightning Network is ready.
- **Step 6:** Right-click a node to see its connection details (REST/gRPC endpoints, macaroon paths).

Note: Polar automatically creates a regtest Bitcoin node behind the scenes. You can mine blocks by clicking the **"+"** button at the top of the Polar window.

4. Python Environment Setup

During the bootcamp we will write Python scripts to query your Bitcoin and Lightning nodes via their RPC and REST interfaces. This section covers installing Python and the libraries you will need.

4.1 Installing Python

macOS

macOS

macOS may ship with Python 3 already. Check your version:

```
python3 --version
```

If you get an error or the version is below 3.10, install via Homebrew:

```
brew install python@3.12
```

Verify:

```
python3 --version # Should show 3.12.x or higher
```

Linux

Linux

Most Linux distributions come with Python 3. Verify:

```
python3 --version
```

If it is missing or too old:

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

Windows — Option A: Inside WSL (if using WSL for Bitcoin Core)

Windows

If you built Bitcoin Core inside WSL, install Python there too so your scripts can talk to your node without cross-platform networking issues:

```
python3 --version
```

If it is missing or too old, install it the same way as Linux:

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

Windows — Option B: Native Windows (for VS Code / IDE users)

Windows

If you prefer to write and run Python scripts from a native Windows IDE like **VS Code**, **PyCharm**, or another editor, install Python natively on Windows as well. This is useful if you want full IDE integration (IntelliSense, debugging, etc.) while your Bitcoin node runs in WSL.

Step 1: Download the installer

- Go to **https://www.python.org/downloads/** and download the latest Python 3.12 installer for Windows.

Step 2: Run the installer

> **Warning:** On the first installer screen, **check the box "Add python.exe to PATH"** before clicking "Install Now". This is critical — without it, `python` and `pip` will not work from PowerShell or Command Prompt.

Step 3: Verify in a new PowerShell window

```
python --version
pip --version
```

> **Note:** On native Windows the command is `python` (not `python3`). If you see a Microsoft Store prompt instead, the PATH was not set correctly — re-run the installer and check the PATH box.

Step 4: Install pip (if not bundled)

Modern Python installers include pip. If `pip --version` fails, install it manually:

```
python -m ensurepip --upgrade
```

> **Note: Connecting native Windows Python to your WSL Bitcoin node:** Your WSL node listens on `127.0.0.1:18443` by default. Native Windows can reach WSL services on `localhost`, so your Python scripts will connect to `http://127.0.0.1:18443` from either environment.

4.2 Virtual Environment and Libraries

macOS Linux Windows

Create a dedicated virtual environment for the bootcamp to keep dependencies isolated:

Create and activate the virtual environment

```
# macOS / Linux / WSL
python3 -m venv ~/bootcamp-env
source ~/bootcamp-env/bin/activate

# Native Windows (PowerShell)
python -m venv $HOME\bootcamp-env
```

```
$HOME\bootcamp-env\Scripts\Activate.ps1
```

Note: If using VS Code with WSL, open the integrated terminal in WSL mode (click the **Remote** icon in the bottom-left corner of VS Code and select "Connect to WSL") to use the Linux virtual environment directly.

Note: You should see `(bootcamp-env)` in your terminal prompt when the environment is active.

Install required libraries

With the virtual environment active, install the following packages:

```
pip install python-bitcoinlib requests
```

Here is what each library is used for:

Library	Purpose
`python-bitcoinlib`	Full-featured Python interface to Bitcoin data structures and RPC
`requests`	HTTP client for querying Lightning node REST APIs

Verify the installation

```
python3 -c "import bitcoin; print('python-bitcoinlib OK')"
python3 -c "import requests; print('requests OK')"
```

5. Troubleshooting

Bitcoin Core Issues

"bitcoin-cli: command not found"

The Bitcoin Core binaries are not on your PATH. Check the installation directory and add it to your shell profile (~/.bashrc, ~/.zshrc, or Windows PATH environment variable). See the platform-specific instructions in Section 2.

"Could not connect to the server" when using bitcoin-cli

This means bitcoind is not running. Start it with:

```
bitcoind -regtest -daemon
```

Then wait a few seconds and try your command again.

"Wallet file verification failed" or wallet errors

If the wallet was created in a different mode or is corrupted, create a new one:

```
bitcoin-cli -regtest createwallet "new_wallet"
```

CMake build fails or missing dependencies

If cmake -B build fails, you are likely missing a dependency. Re-run the install command for your platform (Section 2) and try again. Run the following to see all cmake options:

```
cmake -B build -LH
```

WSL Issues (Windows)

"wsl --install" does nothing or fails

Make sure you are running PowerShell as Administrator. If WSL was previously installed, try updating it:

```
wsl --update
```

Build fails under /mnt/c/ or /mnt/d/

The Bitcoin Core source code must live on the native Linux filesystem inside WSL, not on the Windows-mounted drives. Move your source to your home directory:

```
mv /mnt/c/Users/you/bitcoin ~/bitcoin
cd ~/bitcoin
```

Permission errors in WSL

If you get permission errors, make sure you own the directory:

```
sudo chown -R $USER:$USER ~/bitcoin
```

Polar / Docker Issues

Polar nodes stuck on "Starting"

- Make sure Docker Desktop (macOS/Windows) is running, or Docker Engine (Linux) is active.
- Try restarting Docker: close and reopen Docker Desktop, or run `sudo systemctl restart docker` on Linux.
- If images are missing, Polar will try to pull them. Ensure you have an internet connection.

"permission denied" when running Docker on Linux

You need to be in the docker group:

```
sudo usermod -aG docker $USER
```

Then **log out and log back in** for the group change to take effect.

Docker containers use too much memory

In Docker Desktop, go to Settings and reduce the memory limit. For the bootcamp, 4 GB allocated to Docker should be sufficient.

Python Issues

"No module named 'bitcoin'"

Make sure your virtual environment is activated and run:

```
pip install python-bitcoinlib
```

Python version too old

We require Python 3.10 or newer. Check your version and upgrade if necessary using the instructions in Section 4.1.

Windows (WSL): Python not found

Inside WSL, install Python with:

```
sudo apt install -y python3 python3-pip python3-venv
```

6. Pre-Bootcamp Checklist

Go through each item below and confirm it works on your machine before the bootcamp begins. If you get stuck on any item, reach out to the instructor or post in the bootcamp chat.

Plan A: Bitcoin Core (Built from Source)

- ■ Source code cloned: `git clone https://github.com/bitcoin/bitcoin.git`
- ■ Build completed without errors: `cmake -B build && cmake --build build`
- ■ `bitcoin-cli -regtest --version` prints a version number
- ■ bitcoind starts in regtest mode without errors
- ■ `bitcoin-cli -regtest getblockchaininfo` returns `"chain": "regtest"`
- ■ A wallet has been created and blocks can be mined
- ■ `bitcoin-cli -regtest getbalance` shows a non-zero balance
- ■ (Windows only) All of the above works inside WSL Ubuntu

Plan B: Polar Lightning

- ■ Docker is installed and `docker --version` works
- ■ Docker is running (Docker Desktop shows green, or `docker ps` works on Linux)
- ■ Polar is installed and launches without errors
- ■ A test network has been created with at least 1 Bitcoin node and 2 Lightning nodes
- ■ All nodes show green status in Polar
- ■ You can mine a block using the Polar UI

Python Environment

- ■ `python3 --version` (or `python --version` on native Windows) shows 3.10+
- ■ Virtual environment is created and can be activated
- ■ `pip install python-bitcoinlib requests` completes without errors
- ■ `python3 -c "import bitcoin; import requests; print('All good!')"` prints "All good!"
- ■ (Windows with native Python) `python --version` and `pip --version` work in PowerShell
- ■ (Windows with native Python) Can connect to WSL node at `http://127.0.0.1:18443`

You're all set! If every box above checks out, you are ready for the bootcamp. See you there!